MACHINE LEARNING FUNDAMENTALS

[Document subtitle]

[DATE]

[COMPANY NAME]

[Company address]

Table of Contents

1. What Machine Learning Really Is 4
   1.1 What is Machine Learning? 4
   1.2 Why Machine Learning Works 5
   1.3 ML vs Statistics vs Rule-Based Systems 5
      Rule-Based Systems 6
      Statistics 6
      Machine Learning 6
   1.4 When NOT to Use Machine Learning 7
      1. The rules are known exactly 7
      2. Data is extremely small 7
      3. Interpretability is critical 7
      4. Cost of mistakes is extremely high 7
2. Types of Machine Learning 7
   2.1 Supervised Learning 7
      Regression 8
      Classification 8
   2.2 Unsupervised Learning 8
      Clustering 9
      Dimensionality Reduction 9
   2.3 Semi-Supervised Learning 9
   2.4 Self-Supervised Learning 9
   2.5 Reinforcement Learning 10
   2.6 Batch Learning vs Online Learning 10
      Batch Learning 10
      Online Learning 11
3. ML Problem Formulation 11
   3.1 Regression vs Classification 11
   3.2 Binary vs multi-Class vs multi-label 11
      Binary Classification 11
      Multi-Class Classification 11
      Multi-Label Classification 12
   3.3 Clustering vs Density Estimation 12

Clustering 12
Density Estimation 12
3.4 Anomaly Detection 12
3.5 Ranking and Recommendation 12
4. Train / Validation / Test Split 13
4.1 Why Splitting Data Matters 13
4.2 Train / Validation / Test Sets 13
4.3 Data Leakage 13
4.4 Hold-Out Validation 14
4.5 Cross Validation 14
4.6 When Test Data Should Be Used 14
5. Model Concept 14
5.1 What is a Model? 14
5.2 Parameters vs Hyperparameters 15
5.3 Hypothesis Space 15
5.4 Decision Boundaries 15
5.5 Linear vs Nonlinear Models 16
6. Loss Functions 16
6.1 What is a Loss Function? 16
6.2 Loss vs Metric vs Error 16
6.3 Regression Losses 17
MAE 17
MSE 17
6.4 Classification Loss 17
6.5 Why Accuracy is Not Used for Training 17
7. Evaluation Metrics 17
7.1 Regression Metrics 17
MAE 17
MSE 17
RMSE 17
R^2 17
7.2 Classification Metrics 17
Accuracy 17

Precision 18
Recall 18
F1 Score 18
7.3 Why Accuracy is Often Misleading 18
8. Bias–Variance Tradeoff 18
8.1 Bias 18
8.2 Variance 19
8.3 Underfitting vs Overfitting 19
8.4 Model Complexity vs Data 19
9. End-to-End ML Workflow 19
9.1 Problem Definition 19
9.2 Data Collection 19
9.3 Data Understanding 20
9.4 Train–Validate Loop 20
9.5 Model Selection 20
9.6 Final Evaluation 20
9.7 Deployment Awareness 20
10. Common ML Pitfalls 20
10.1 Data Leakage 20
10.2 Improper Evaluation 21
10.3 Ignoring Baseline Models 21
10.4 Blindly Trusting Metrics 21

1. What Machine Learning Really Is

1.1 What is Machine Learning?

Machine Learning is a subfield of **Artificial Intelligence** that focuses on building systems capable of **learning patterns from data and making predictions or decisions without being explicitly programmed for every rule**.

Traditional software works by writing **explicit rules**.

Example:

A programmer writes rules for calculating tax.

```
if income > 500000:
   tax = income * 0.30
else:
   tax = income * 0.10
```

This works when the rules are **clear and deterministic**.

However, many real-world problems do not have explicit rules.

Examples:

- Identifying spam emails
- Predicting stock prices
- Diagnosing diseases from medical images
- Recognizing speech or faces

For these problems, the rules are **too complex or unknown**.

Machine Learning solves this by:

1. Collecting data
2. Training an algorithm on that data
3. Letting the algorithm **discover patterns automatically**

Conceptually:

Data + Learning Algorithm → Model
Model + New Input → Prediction

Example:

Training data:

Size (sqft)	Rooms	Price
1000	2	5L
1500	3	7L
2000	4	10L

The algorithm learns the **relationship between features and price**.

Once trained, it can estimate price for a new house.

1.2 Why Machine Learning Works

Machine learning works because **real-world data contains structure**.

This structure often comes from **underlying causal processes**.

Example:

House prices are influenced by:

- size
- location
- number of rooms
- age of property

Even if the relationship is complex, the data still contains **patterns**.

Machine learning algorithms attempt to approximate an unknown function:

$y = f(x)$

Where:

x = input features
y = output
f = unknown relationship

The model attempts to **approximate this function** using data.

Example:

$f(\text{size}, \text{rooms}, \text{location}) \approx \text{price}$

More data usually means the algorithm can estimate this function more accurately.

This is why **data quality and quantity matter so much**.

1.3 ML vs Statistics vs Rule-Based Systems

These fields overlap but have different goals.

Rule-Based Systems

Rule-based systems rely entirely on **human-defined logic**.

Example:

Spam filter rules:

```
if email contains "free money":
    spam
```

Problems:

- rules become huge
- maintenance becomes impossible
- fails with new patterns

Statistics

Statistics focuses on:

- modeling uncertainty
- explaining relationships
- inference about populations

Example:

Testing whether **smoking causes cancer**.

Statistical models prioritize **interpretability and theory**.

Machine Learning

Machine learning focuses on:

- prediction
- scalability
- performance on unseen data

Modern ML combines:

- statistics
- optimization
- computer science

In practice, ML models are **statistical models trained using optimization algorithms**.

1.4 When NOT to Use Machine Learning

ML is powerful but **not always appropriate**.

Avoid ML when:

1. The rules are known exactly

Example:

area = length × width

A simple formula is better.

2. Data is extremely small

Machine learning depends heavily on **large datasets**.

With very small data, models will **overfit**.

3. Interpretability is critical

Some ML models (especially deep learning) are **black boxes**.

In domains like medicine or law, interpretability may be required.

4. Cost of mistakes is extremely high

Example:

Nuclear reactor control systems rely on **deterministic algorithms**, not ML.

2. Types of Machine Learning

2.1 Supervised Learning

Supervised learning is the most common ML paradigm.

The dataset contains:

Input features (X)
Output labels (Y)

Goal:

Learn mapping:

$f: X \rightarrow Y$

Example:

Spam detection.

Dataset:

Email Text	Label
Free money now	Spam
Meeting tomorrow	Not spam

The algorithm learns patterns connecting **text features → spam label**.

Supervised learning tasks are mainly divided into:

Regression

Predict continuous values.

Example:

- house prices
- temperature prediction
- demand forecasting

Output:

Real number

Classification

Predict discrete categories.

Example:

- disease detection
- spam filtering
- image classification

Output:

Class label

2.2 Unsupervised Learning

Unsupervised learning deals with **unlabeled data**.

There is **no target variable**.

Goal:

Find hidden patterns in the data.

Clustering

Group similar items together.

Example:

Customer segmentation.

Customers may be grouped by:

- purchasing behavior
- demographics
- spending patterns

Dimensionality Reduction

Real datasets may contain **hundreds of features**.

Dimensionality reduction compresses data into fewer dimensions while preserving structure.

Example:

PCA (Principal Component Analysis).

2.3 Semi-Supervised Learning

In real-world scenarios:

- labeled data is expensive
- unlabeled data is abundant

Example:

Medical imaging datasets.

Only a small portion of images are labeled by doctors.

Semi-supervised learning combines:

Small labeled dataset
+
Large unlabeled dataset

This often improves performance significantly.

2.4 Self-Supervised Learning

Self-supervised learning generates **labels automatically from data**.

Example in natural language processing:

Sentence:

The cat sat on the ___

The model learns to predict the missing word.

No human labeling required.

Modern AI systems (including large language models) rely heavily on this approach.

2.5 Reinforcement Learning

Reinforcement learning models' **sequential decision making**.

Key components:

COMPONENT	MEANING
AGENT	Learner
ENVIRONMENT	External system
ACTION	Decision taken
REWARD	Feedback signal

Goal:

Maximize long-term cumulative reward.

Example:

Training a robot to walk.

The agent learns by:

trial → feedback → improvement

Applications:

- robotics
- game AI
- recommendation systems
- autonomous driving

2.6 Batch Learning vs Online Learning

Batch Learning

Model trains on the **entire dataset at once**.

Typical pipeline:

collect data → train model → deploy → retrain later

Example:

Predicting house prices using a fixed dataset.

Online Learning

Model learns **incrementally from streaming data**.

Example:

Real-time recommendation systems.

The model updates continuously as new data arrives.

3. ML Problem Formulation

Before training a model, the problem must be **mathematically defined**.

3.1 Regression vs Classification

Regression

Output is **continuous**.

Examples:

- predicting income
- predicting energy consumption

Classification

Output is **categorical**.

Examples:

- fraud detection
- disease classification

3.2 Binary vs multi-Class vs multi-label

Binary Classification

Two classes.

Example:

fraud / not fraud

Multi-Class Classification

More than two classes.

Example:

apple / banana / orange

Multi-Label Classification

Each example may have **multiple labels simultaneously**.

Example:

Image tagging:

beach + sunset + ocean

3.3 Clustering vs Density Estimation

Clustering

Group similar data points.

Example:

Segmenting customers by buying habits.

Density Estimation

Estimate probability distribution of data.

Used in:

- anomaly detection
- generative models

3.4 Anomaly Detection

Goal:

Detect **rare events or abnormal behavior**.

Examples:

- credit card fraud
- network intrusion
- manufacturing defects

These events occur rarely but are extremely important.

3.5 Ranking and Recommendation

Important in many internet systems.

Examples:

- Google search ranking
- YouTube recommendations
- Amazon product ranking

Instead of predicting labels, the model predicts **relative ordering**.

4. Train / Validation / Test Split

4.1 Why Splitting Data Matters

If a model is tested on the same data it was trained on, it may simply **memorize the data**.

This gives an **illusion of high performance**.

Example:

Model memorizes every training example.

Training accuracy = 100%.

But new data accuracy may be **very poor**.

This is why we must evaluate on **unseen data**.

4.2 Train / Validation / Test Sets

Typical split:

Dataset	Role
Train	Learn model parameters
Validation	Tune hyperparameters
Test	Final unbiased evaluation

Example split:

70% training
15% validation
15% testing

4.3 Data Leakage

Data leakage is one of the **most dangerous mistakes in ML**.

It occurs when **information from the future or test set leaks into training**.

Example:

Suppose we predict loan default.

If the dataset includes a feature:

loan_recovery_status

This information is only known **after default occurs**.

Including it during training creates **unrealistic models**.

4.4 Hold-Out Validation

Simplest evaluation method.

Split dataset once into train/validation/test.

This method is fast but may produce **unstable results** if the dataset is small.

4.5 Cross Validation

Cross-validation improves evaluation reliability.

Example: **5-fold cross validation**

Dataset divided into 5 parts.

Process:

1. Train on 4 parts
2. Validate on remaining part
3. Repeat 5 times

Final performance is the **average of all runs**.

This reduces evaluation variance.

4.6 When Test Data Should Be Used

Test data must remain **untouched during model development**.

It is only used **once**:

final evaluation

Using test data repeatedly leads to **biased models**.

5. Model Concept

5.1 What is a Model?

A model is a **mathematical function approximating the relationship between inputs and outputs**.

Example:

Linear regression model:

price = w1 × size + w2 × rooms + b

Where:

- w1, w2 = weights
- b = bias

Training means **learning these parameters from data**.

5.2 Parameters vs Hyperparameters

Parameters

Learned automatically during training.

Examples:

- neural network weights
- regression coefficients

Hyperparameters

Defined before training.

Examples:

- number of trees
- learning rate
- number of layers

These control **how the model learns**.

5.3 Hypothesis Space

Hypothesis space = **all possible models an algorithm can represent**.

Example:

Linear regression can represent **all linear functions**.

Neural networks can represent **very complex nonlinear functions**.

Choosing the right hypothesis space is critical.

5.4 Decision Boundaries

In classification, the model divides feature space into regions.

These regions are separated by **decision boundaries**.

Example:

A straight line separating two classes.

More complex models produce **curved boundaries**.

5.5 Linear vs Nonlinear Models

Linear models assume relationships are **linear combinations of features**.

Example:

$y = w1x1 + w2x2 + b$

Nonlinear models capture complex relationships.

Examples:

- decision trees
- neural networks
- kernel methods

6. Loss Functions

6.1 What is a Loss Function?

Loss measures **how wrong a prediction is**.

Example:

If true price is 10 and prediction is 8:

error = 2

Loss functions convert errors into **optimization objectives**.

Goal:

minimize loss

6.2 Loss vs Metric vs Error

TERM	PURPOSE
LOSS	Used during training
ERROR	General deviation
METRIC	Used for evaluation

6.3 Regression Losses

MAE

Average absolute difference.

Less sensitive to outliers.

MSE

Squares errors before averaging.

Large errors become **very costly**.

6.4 Classification Loss

Cross-entropy loss evaluates **probability predictions**.

A confident wrong prediction receives **very high penalty**.

6.5 Why Accuracy is Not Used for Training

Accuracy is **not differentiable**.

Optimization algorithms require **smooth gradients**.

Loss functions provide this.

7. Evaluation Metrics

7.1 Regression Metrics

MAE

Average absolute prediction error.

MSE

Penalizes large errors.

RMSE

Square root of MSE.

R^2

Measures proportion of variance explained.

7.2 Classification Metrics

Accuracy

Fraction of correct predictions.

Precision

Measures correctness of positive predictions.

Recall

Measures ability to detect positive cases.

F1 Score

Harmonic mean of precision and recall.

Useful when classes are imbalanced.

Confusion Matrix

Shows prediction breakdown.

	Predicted Positive	**Predicted Negative**
Actual Positive	TP	FN
Actual Negative	FP	TN

This matrix reveals **exact types of errors**.

7.3 Why Accuracy is Often Misleading

Consider fraud detection:

99.5% normal transactions
0.5% fraud

A model predicting **all transactions as normal** achieves:

99.5% accuracy

But it detects **zero fraud cases**.

Thus precision/recall are more informative.

8. Bias–Variance Tradeoff

8.1 Bias

Bias measures **systematic error due to model simplicity**.

High bias → underfitting.

Example:

Linear model on nonlinear data.

8.2 Variance

Variance measures **model sensitivity to training data changes**.

High variance → overfitting.

The model memorizes noise.

8.3 Underfitting vs Overfitting

Underfitting:

Model too simple.

Overfitting:

Model too complex.

Goal:

balanced complexity

8.4 Model Complexity vs Data

Complex models require **more data**.

Otherwise they overfit easily.

9. End-to-End ML Workflow

9.1 Problem Definition

Define:

- target variable
- constraints
- evaluation metric

9.2 Data Collection

Data sources include:

- databases
- APIs
- sensors
- logs

9.3 Data Understanding

Engineers perform:

- exploratory data analysis
- missing value analysis
- feature distribution checks

9.4 Train–Validate Loop

The model is repeatedly improved through:

train → evaluate → adjust → retrain

9.5 Model Selection

Multiple algorithms are tested.

Best model selected based on:

- performance
- robustness
- interpretability

9.6 Final Evaluation

Evaluate on **test set only once**.

This estimates **real-world performance**.

9.7 Deployment Awareness

Production ML systems require:

- monitoring
- retraining
- handling data drift

10. Common ML Pitfalls

10.1 Data Leakage

Occurs when training uses **information unavailable during inference**.

This produces unrealistic performance.

10.2 Improper Evaluation

Common mistakes:

- testing on training data
- tuning on test data

10.3 Ignoring Baseline Models

Always compare against simple models.

Example baselines:

- linear regression
- logistic regression

10.4 Blindly Trusting Metrics

Metrics alone cannot guarantee model quality.

Engineers must inspect:

- data distribution
- prediction errors
- real-world impact